Here's how this script maps directly onto the RAG pattern, step by step, using only the code you provided.

RAG has three core phases:
1) Chunk & embed your knowledge base
2) Retrieve relevant chunks for a query
3) Generate an answer with an LLM using retrieved context

Below is how each phase appears in this file.

---

1) RAG Step 1: Chunk the corpus into retrievable pieces

Concept:
RAG doesn't feed whole documents to the model; it splits them into smaller, searchable units so retrieval can be precise and token usage stays reasonable. This script uses a very simple paragraph-level split to show how chunking choices affect what can be retrieved together.

Code + comments:

```
def chunk_corpus() -> list[dict]:
    chunks = []
    for path in sorted(CORPUS.glob("*.md")):   # scan all docs
        text = path.read_text()
        for para in re.split(r"\n\s*\n", text):  # split on blank lines (paragraphs)
            para = para.strip()
            if len(para) < 40:  # skip tiny fragments/headers
                continue
            chunks.append({"source": path.name, "text": para})
    return chunks
```

Key points:
- No fancy overlap or sliding windows.
- Intentionally crude to prove that chunking policy directly limits what RAG can ever find in one shot.

---

2) RAG Step 1 (continued): Build the vector index

Concept:
Each chunk is converted into a dense embedding (vector). These vectors form your "index". In production this lives in a specialized DB; here it's just a NumPy array to expose that RAG mathematically is similarity search over embeddings.

Code + comments:

```
def build_index(chunks: list[dict]) -> np.ndarray:
```

```
    # Encode all chunks with the same model used at query time
    vectors = encoder().encode(
        [c["text"] for c in chunks],
        normalize_embeddings=True  # important: enables dot-product as cosine
similarity
    )
    return np.asarray(vectors)
```

Key points:
- `normalize_embeddings=True` means all chunk vectors lie on the unit sphere.
- The resulting array is your entire "vector database": each row = one chunk's semantics.

---

3) RAG Step 2: Retrieve relevant chunks for a query

Concept:
Given a user question, embed it with the same model and find the closest chunks by cosine similarity. Those become the context injected into the LLM prompt.

Code + comments:

```
def retrieve(query: str, chunks, index, k=3):
    # 1) Embed the user query (same space as chunk vectors)
    qvec = encoder().encode([query], normalize_embeddings=True)[0]

    # 2) Compute similarity in one shot: normalized dot product == cosine similarity
    scores = index @ qvec

    # 3) Take top-k most similar chunks
    top = np.argsort(scores)[::-1][:k]

    # 4) Return matched chunks plus their similarity score
    return [{**chunks[i], "score": float(scores[i])} for i in top]
```

Key points:
- No external vector DB; retrieval is literally a matrix-vector multiply.
- Top-k (default 3) embodies RAG's fundamental bias: only local, highest-similarity context is visible to the model.

---

4) RAG Step 3: Generate with an LLM using retrieved context

Concept:
RAG "augments" the prompt: you give the LLM both the question and the relevant excerpts, then instruct it to answer strictly from that material. This reduces hallucination while grounding responses in your documents.

Code + comments:

PROMPT = """Answer the question using only the context below. If the context does not contain enough information to answer, say so explicitly rather than guessing.

Context:
{context}

Question: {question}"""

```
def answer(question, retrieved):
    # Assemble context from retrieved chunks (with source labels)
    context = "\n\n---\n\n".join(
        f"[{c['source']}]\n{c['text']}" for c in retrieved
    )

    # Call the LLM with question + context
    resp = client.messages.create(
        model=MODEL,
        max_tokens=600,
        messages=[{"role": "user",
                   "content": PROMPT.format(context=context, question=question)}],
    )

    # Extract only text parts from the response
    return "".join(b.text for b in resp.content if b.type == "text")
```

Key points:
- The prompt enforces faithfulness: “Answer using only the context below.”
- If RAG retrieval fails to bring relevant chunks, the model is forced to say so instead of guessing—this exposes fragility when retrieval is insufficient.

---

5) How it’s wired together (orchestration)

Concept:
The script shows the full RAG flow end-to-end and then deliberately tests its limits.

Relevant bits + comments:

```
if __name__ == "__main__":
    # CLI setup: question, top-k, debug flags
    ap = argparse.ArgumentParser()
    ap.add_argument("question", nargs="?")
    ap.add_argument("-k", type=int, default=3)
    ap.add_argument("--show-chunks", action="store_true")
    ap.add_argument("--failures", action="store_true")
```

```
    args = ap.parse_args()

    # 1) Build chunks and index (the RAG knowledge base)
    chunks = chunk_corpus()
    index = build_index(chunks)

    if args.failures:
        # Special mode: run queries known to break naive RAG
        run_failures(chunks, index, args.show_chunks)
    else:
        # Normal RAG flow for a single question
        q = args.question or "how many days annual leave do I get?"
        retrieved = retrieve(q, chunks, index, args.k)

        # Show what was retrieved (diagnostics)
        for c in retrieved:
            preview = c["text"][:80].replace("\n", " ")
            print(f"  {c['score']:.3f}  {c['source']:<24} {preview}...")

        # Generate the final answer using RAG context
        print(answer(q, retrieved))
```

Key points:
- Normal run = textbook RAG: index → retrieve top-k → generate.
- `--failures` mode runs three adversarial questions to show where this exact pattern fails:
  - multi-hop reasoning across files
  - subtle negations
  - requiring global aggregation beyond top-k chunks

If you'd like, I can next walk through one of those "failure" queries and show concretely how changing the chunking strategy or retrieval logic could partially fix it.